

# CCAGENT: Coordinating Collaborative Data Scaling for Operating System Agents via Web3

Liang Chen  
National Key Laboratory for  
Multimedia Information Processing,  
Peking University  
Beijing, China  
leo.liang.chen@outlook.com

Haozhe Zhao  
National Key Laboratory for  
Multimedia Information Processing,  
Peking University  
Beijing, China  
mimazhe55360@gmail.com

Yinzhen Huang  
BUAA  
Beijing, China  
22373561\_hyz@buaa.edu.cn

Yang Luo  
BUAA  
Beijing, China  
22371028@buaa.edu.cn

Tsekai Lin  
University of Toronto  
Toronto, Canada  
0xplutoo@gmail.com

Weichu Xie  
Beijing Institute of Technology  
Beijing, China  
1120222370@bit.edu.cn

Ruoyu Wu  
Peking University  
Beijing, China  
roylala726@gmail.com

Peiyi Wang  
Peking University  
Beijing, China  
peiyiwang9979@gmail.com

Runxin Xu  
Deepseek AI  
Beijing, China  
runxinxu@gmail.com

Ming Wu  
Zero Gravity Labs  
Shanghai, China  
ming@0g.ai

Baobao Chang  
Peking University  
Beijing, China  
chbb@pku.edu.cn

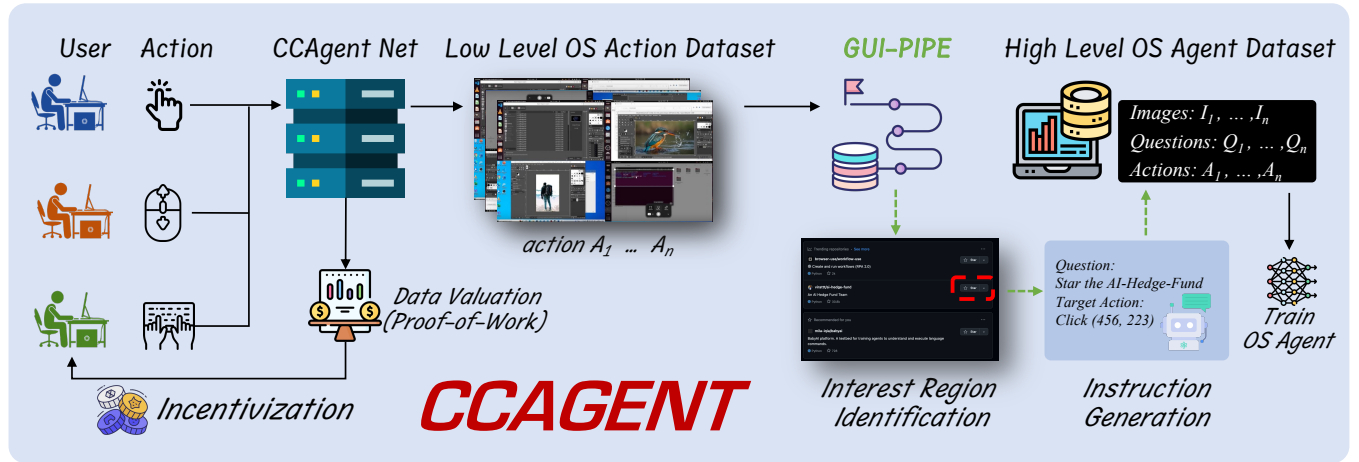


Figure 1: General framework of CCAGENT system including CCAgent Net for dataset collection and GUI-Pipe for post-processing.

## Abstract

The current AI revolution, fueled by Large Language Models (LLMs), heavily relies on vast open-access internet data. However,

the Operating System (OS) Agent field faces a significant data sparsity challenge due to the lack of public data collection systems and privacy concerns. To address this, we introduce CCAgent Net, a system that coordinates and incentivizes internet users to contribute to scaling OS agent datasets. Furthermore, we propose GUI-Pipe, an automated data post-processing pipeline that evaluates, filters, and transforms raw user-uploaded OS data into trainable image-instruction-answer data. This process results in CCAgent-Instruct, the largest instruction-based multi-platform GUI agent dataset. Our



This work is licensed under a Creative Commons Attribution 4.0 International License.  
CIKM '25, Seoul, Republic of Korea  
© 2025 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-2040-6/2025/11  
<https://doi.org/10.1145/3746252.3761392>

experiments demonstrate CCAgent’s effectiveness in advancing OS Agent development. For instance, our CCAgent-GUI-3B model achieves a score of 33.7 (+127%) on the challenging out-of-domain Screenspots-Pro benchmark, significantly outperforming other advanced open-source models like UI-TARS-2B, Qwen2.5-VL-3B/7B, and UGround-V1-7B, even those of larger sizes. Our experiments also reveal the scaling behaviors of GUI Agent training and insights for future direction.

## Keywords

Multimodal LLM, GUI Agent, Data Collection

### ACM Reference Format:

Liang Chen, Haozhe Zhao, Yinzhen Huang, Yang Luo, Tsekai Lin, Weichu Xie, Ruoyu Wu, Peiyi Wang, Runxin Xu, Ming Wu, and Baobao Chang. 2025. CCAgent: Coordinating Collaborative Data Scaling for Operating System Agents via Web3. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM '25)*, November 10–14, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3746252.3761392>

## 1 Introduction

Operating System Agents (OS Agents) [12] represent a significant frontier in automating complex tasks. These agents operate directly within the user interfaces (e.g., GUIs) of computing devices such as computers and smartphones, enabling them to execute diverse workflows. Enabled by the revolutionary advancements in Multimodal Large Language Models (MLLMs), we have witnessed remarkable progress in OS Agent development, exemplified by pioneering systems like UI-TARS [28] and Claude-Compute-Use [1].

However, a critical disparity emerges when comparing the flourishing landscape of LLM development with the nascent field of OS Agents. The current AI revolution, largely driven by LLMs [26, 31], is built upon the bedrock of massive open-access data from the internet, including sources like CommonCrawl [7] and platforms such as GitHub. This continuous influx of diverse data—spanning text, code, images, and videos—continually fuels the advancement of more advanced LLMs. In stark contrast, despite trillions of daily computer and smartphone interactions globally, the corresponding action data necessary for robust OS Agent training is rarely collected, and publicly available datasets are exceedingly scarce. This severe data hunger problem significantly impedes the widespread development and deployment of truly capable and open OS Agents.

Based on preliminary statistical experiments, we’ve found that users perform about 3,000 atomic computer operations per hour in work-related scenarios, far exceeding the volume of textual data a single person can produce. Given billions of global OS users, this could generate trillions of operational data points daily. However, this vast amount of real user action data has not been systematically collected to train OS agents, primarily due to privacy concerns and the lack of robust data processing pipelines. The largest available open datasets in the research community (e.g., AITW [30], Insight-UI [39]) are limited to millions of data points and are mostly from simulations, not genuine user interactions, highlighting a significant gap between the generated data and what is being used for research.

This raises an intriguing question: **Can we build a system to collect the vast real user interaction data generated daily by**

**real users worldwide, and establish an automated pipeline for data processing and model training to enhance OS agents?**

To address this challenge, we introduce **CCAgent**, a comprehensive system illustrated in Figure 1. CCAgent comprises two core components: CCAgent-Net, a novel web3-based GUI data collection system, and GUI-Pipe, an automated post-processing pipeline.

**CCAgent-Net** is a distributed operating system action collection system that leverages Web3’s incentivization mechanisms. It gathers low-level human-computer interaction data such as click, type and scroll with corresponding screenshot, performing data verification on the user’s edge machine before server upload. The server then conducts further verification and employs a data valuation algorithm to ensure data quality and allocate compensation to users, all without human intervention. Complementing this, **GUI-Pipe** is a data processing pipeline designed to transform the low-level action data collected by CCAgent-Net into trainable multimodal instruction tuning data, specifically for enhancing OS agents.

Using the CCAgent system, we collected 1 million low-level operation data points from users and subsequently generated 100 million instruction tuning data instances via GUI-Pipe. We then fine-tuned a relatively small OS agent model, Qwen2.5-VL-3B, using a 1 million subset of this generated data. This resulted in consistent and significant performance gains on both in-domain (+38.3, +84.1%) and challenging out-of-domain benchmarks like ScreenSpot-pro (+19.0, +127%). Notably, our fine-tuned model outperformed strong baseline models that were larger and utilized extensive in-house data, thereby validating the efficacy of the CCAgent system and the GUI-Pipe pipeline. We plan to open-source the code, model, and license data usage to foster future research in the OS agent field.

## 2 Related Work

*Model.* Early OS-agent models, such as WebGPT [24], utilized large language models (LLMs) fine-tuned via behavior cloning and reinforcement learning within text-browser environments. Subsequent methods, including Mind2Web [8], advanced these techniques by combining smaller, fine-tuned language models for content filtering with larger LLMs for action prediction. Recent graphical user interface (GUI) agents have increasingly favored vision-based methods. OmniParser [18] and SeeClick [6] parse screenshots into structured UI elements, facilitating subsequent interaction. Initial OS-agent frameworks, such as OS-Copilot [33] and AIOS [22], treated LLMs as task decomposers interfacing across applications. Although effective for task-specific deployments, these modular systems relied on manual optimization and specialized components, limiting scalability and generalization compared to end-to-end models [29]. To address these limitations, recent researches have shifted towards purely vision-based models that operate exclusively on screenshots, eliminating dependencies on textual interfaces like HTML [13]. Additionally, researchers have moved away from these modular, specialized-model architectures [18, 33, 37] to unified, end-to-end agent models, that directly interact with the GUI of system, such as CogAgent [11], ShowUI [17], OS-Atlas [34] and Aria-UI [35]. These unified architectures incorporate traditionally separate modules within a single LLM-driven framework [10], facilitating seamless information flow and comprehensive task execution. Models

**Table 1: Summary of Datasets for OS Agents**

| Dataset Name             | Domain           | Purpose              | Data Type                              | Number of Data |
|--------------------------|------------------|----------------------|----------------------------------------|----------------|
| Android in the Wild [30] | Mobile (Android) | Training, Evaluation | Human demos (actions, NL instructions) | 715K           |
| OmniACT [36]             | Desktop, Web     | Evaluation           | Human demos (screenshots, NL, code)    | 9.8K           |
| Mind2Web [8]             | Web              | Training, Evaluation | Human demos (NL instructions, actions) | 2.3K           |
| Explorer [21]            | Web              | Training             | Synthetic (web navigation)             | 94K            |
| Insight-UI [39]          | Cross-platform   | Training             | Synthetic (UI interaction)             | 1.46M          |
| ScreenSpot-Pro [16]      | Desktop          | Evaluation           | Human demos (UI grounding)             | 1.6k           |
| AndroidInteraction [14]  | Mobile (Android) | Evaluation           | Annotated logs (agent interaction)     | 750            |
| MobileSafetyBench [4]    | Mobile (Android) | Evaluation           | Synthetic/human (safety tasks)         | 80             |
| GUI-WORLD [9]            | General GUI      | Training, Evaluation | Video annotations, actions, QA         | 12.3K          |
| <b>CCAgent (Ours)</b>    | Desktop OS       | Training, Evaluation | Automated logs (actions, screenshots)  | >100M          |

such as UI-TARS [29] and UI-R1 [20] exemplify this trend, achieving superior performance by refining perceptual capabilities and optimizing action predictions through reinforcement learning.

**Dataset.** The development of Operating System (OS) agents requires diverse datasets tailored to specific platforms and tasks. **Human Demonstrations**, used in datasets like Android in the Wild (AitW) [30] and OmniACT [36], record goal-oriented actions with natural language instructions, ideal for supervised learning but less scalable. **Synthetic Generation**, seen in Explorer [21] and Insight-UI [39], creates controlled data for edge cases, though it may miss real-world nuances. **Reorganization and Annotation**, as with AndroidInteraction [14], repurposes existing data with task-specific annotations, constrained by original data scope. Annotations vary from implicit action logs to explicit labels like goal instructions (Mind2Web [8], AitW [30]), UI grounding (ScreenSpot-Pro [16]), code generation (OmniACT [36]), task success (Explorer [21]), safety (MobileSafetyBench [4]), or video (GUI-WORLD [9]). Trends show specialization by platform (Web, Mobile, Desktop), capability (grounding, safety, code), and domain, with efforts like OmniACT [36] and CRAB [32] bridging environments. Despite these developments, current datasets face notable limitations. As summarized in Figure 4, human-annotated datasets, while high in quality, are expensive and limited in scale—typically ranging from 1k to 700k samples. Synthetic datasets offer greater scalability but often fall short in realism and diversity. In contrast, CCAgent introduces a novel approach by integrating automated data augmentation with diverse real-world data sources. This enables the construction of the largest OS-agent dataset to date, achieving broad coverage and diversity while maintaining high quality.

### 3 Preliminaries

#### 3.1 Problem Statement

Graphical User Interface (GUI) agent [25] is a type of OS agent, which generally refers to the task of aligning user instructions, typically in natural language, with corresponding actionable elements within a GUI environment. Formally, given a dataset of GUI grounding instances, each instance consists of a screenshot  $I_k$  representing the GUI state and a corresponding natural language question  $Q_k$ . The objective is to predict the action sequence  $A_k = (a_1, a_2, \dots, a_\ell)$  that fulfills the intent expressed in  $Q_k$  within the context of  $I_k$ .

- **Input:** A GUI image  $I_k$  (e.g., a screenshot), and a natural language question  $Q_k$  describing the desired task.

- **Output:** An action sequence  $A_k = (a_1, a_2, \dots, a_\ell)$  that completes the task described by  $Q_k$  based on the content of  $I_k$ .

The task requires multimodal reasoning over both the visual structure of the GUI in  $I_k$  and the semantics of the instruction in  $Q_k$ .

#### 3.2 Vision-Language Model and GUI-Agent

A Vision-Language Model (VLM) [5] processes multi-modal input data—typically an image and a natural language prompt—to produce a language-based output. Formally, the model is defined as an autoregressive conditional language model:

$$P(y_t \mid y_{<t}, I, T),$$

where  $I$  is the input image (e.g., a screenshot of the GUI),  $T$  is the textual prompt (e.g., a natural language command or task description), and  $y_t$  is the  $t$ -th token in the output sequence. The model is trained to predict the next token conditioned on the previous tokens, the visual input, and the prompt.

In the context of GUI-Agent, a VLM can serve as the core decision-making engine. The GUI-Agent receives as input:

- Screenshots representing the current GUI state (visual input  $I$ ).
- A natural language task description (text input  $T$ ).

The VLM then outputs a sequence of tokens that represent either a high-level instruction, a structured command, or an actionable plan. These outputs can be interpreted and executed by a downstream controller that maps textual actions to atomic OS-level operations

*Example:* Given a screenshot  $I$  showing a settings window and a prompt  $T$ : “Enable dark mode,” the model generates the executable action including the action type (Click) and the parameter (coordinate: [456,12]) of the action:

[Click, "Dark Mode Toggle", [456, 12] ].

This generative approach offered by VLM allows the GUI-Agent to perform open-ended reasoning, contextual understanding, and action planning using a unified Vision-Language interface.

### 4 CCAgent Net: Scaling GUI Data Collection

The goal of CCAgent Net is to gather a massive, diverse dataset of real-world user interactions with operating systems. This section details CCAgent Net’s design principles, focusing on its architecture and the mechanisms ensuring data integrity and fair compensation.

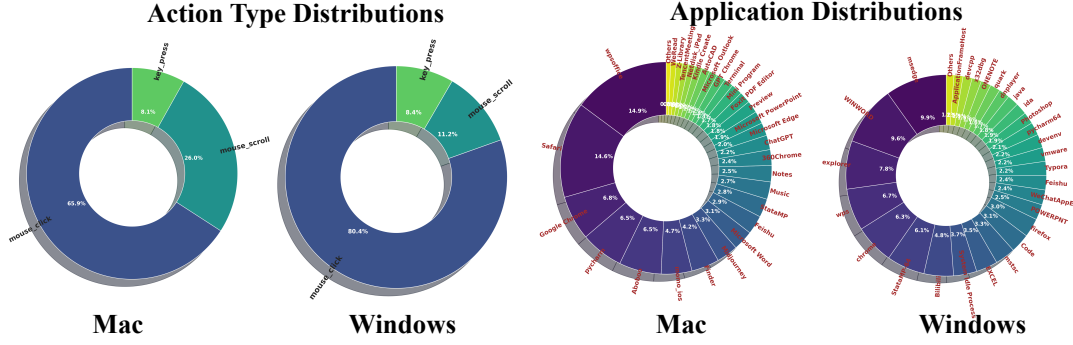


Figure 2: Action Type and Application Distributions of CCAgent Raw Data.

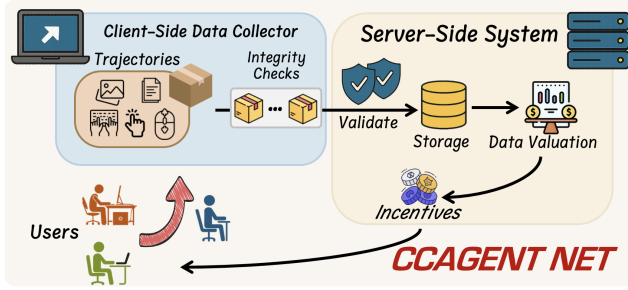


Figure 3: Illustration of CCAgent Net.

#### 4.1 System Overview

As shown in Figure 3, CCAgent Net is a distributed system designed to collect real-time OS action data from a global network of users. Our primary objective is to overcome the current data scarcity in OS agent training by creating an unprecedentedly large and diverse dataset of user interactions. The system comprises two main components: 1) a light-weight client-side data collector deployed on user machines, 2) a robust server-side infrastructure responsible for data validation, valuation, storage for future post-processing and incentive distribution. Inspired by Web3’s incentivization mechanisms, CCAgent Net aims to create a symbiotic relationship where users contribute valuable behavioral data in exchange for tangible rewards, thereby fostering an vibrant, data-contributing community.

#### 4.2 Proof of Work by Uploading Trajectory

Proof of Work (PoW) is a consensus mechanism in which one party (the prover) demonstrates to others (the verifiers) that a significant, but feasible, amount of computational effort has been expended. PoW is widely used in many Web3 systems, with Bitcoin [23] being the most prominent example. In CCAgent Net, we introduce a novel PoW mechanism based on user-generated GUI action trajectories. The primary goal of this system is to effectively balance the need for high-quality data collection with fair user compensation, ensuring that contributions are valuable and rewarded appropriately.

Instead of computationally intensive cryptographic puzzles, a user’s “work” is defined by the valid, real-world sequences of OS

operations they perform and upload. Each trajectory represents a continuous segment of user interaction, encompassing atomic actions like clicks, scrolls, keystrokes, along with their associated contextual metadata (e.g., active application information, timestamp, screenshots). Users are rewarded based on the verified value of these uploaded trajectories, ensuring that compensation is directly tied to the utility and integrity of their contributions to the dataset.

**4.2.1 Data Verification.** Robust data verification is paramount to prevent malicious submissions and maintain the integrity of the CCAgent ecosystem. Our multi-layered verification process includes:

- **Client-Side Integrity Checks:** The CCAgent client employs checksums to ensure that collected trajectories haven’t been tampered with before submission.
- **Server-Side Validation:** Upon receiving a trajectory, the server performs a comprehensive set of automated checks. This includes validating timestamps, detecting impossible action sequences (e.g., a click without a prior mouse movement) and removing data with the same hash code.
- **Behavioral Anomaly Detection:** In the server side, we also implement some heuristic algorithms to identify bot-like patterns (e.g., perfectly uniform delays, repetitive actions). In the future, we plan to leverage machine learning models trained on legitimate user behavior to identify and flag trajectories that deviate significantly from typical human interaction patterns, helping filter out simulated or fabricated data.
- **Safety and Privacy:** The user can choose when to begin and when to stop the recording of their GUI action trajectories, granting them direct control over the data collection process. The user can always view and delete the already uploaded trajectories, making full control of their data, thus fostering trust and ensuring users feel secure while contributing.

Only trajectories that successfully pass these rigorous verification stages are accepted into the dataset and become eligible for valuation.

**4.2.2 Data Valuation.** Once a trajectory is verified, its value is quantitatively assessed to determine the appropriate reward for the contributing user. This valuation process considers several factors

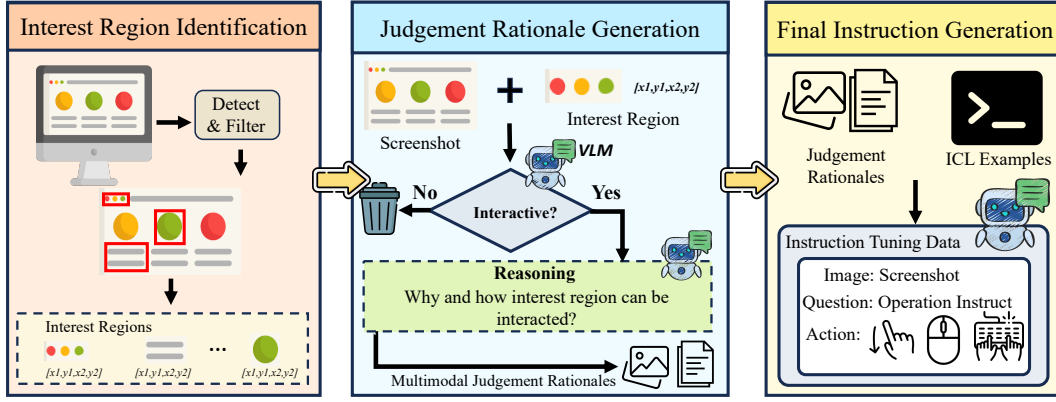


Figure 4: Illustration of GUI-PIPE.

to prioritize data that is most beneficial for training robust OS agents:

- **Trajectory Length and Complexity:** Longer, more intricate sequences of operations that involve multiple applications or complex workflows are generally valued higher than short, simple, or repetitive actions.
- **Action Type:** Trajectories demonstrating a wide range of atomic operations (clicking, typing and scrolling.) are preferred as they offer a richer understanding of user intent and interaction. Note that the continuous typing action are merged to 1 action. We have a fixed rate for each type of action and multiplied by a coefficients related to the diversity of the actions uploads.
- **Application Type:** We manage a in-house application list that has a initial rate table for different applications, the rate is also fluctuated considering the amount of data available currently. More available data on the platform, the rate would be lower. This fluctuated rate design could prevent the users from always uploading the most high rate data, which harms the data diversity.

The value  $V(T)$  of a given trajectory  $T$  can be calculated as a weighted sum of scores derived from its characteristics:

$$V(T) = w_{LC} \cdot S_{LC} + w_A \cdot S_A + w_S \cdot S_S$$

where  $S_{LC}$  represents the score for the trajectory’s length and complexity.  $S_A$  is the diversity score for the action types it contains (like clicks, typing, etc.).  $S_S$  quantifies the rate of the type of application. Each of these scores is multiplied by a predefined respective weight ( $w_{LC}$  for length/complexity,  $w_A$  for action diversity, and  $w_S$  for scenario/application value) to signify its relative importance.

This dynamic valuation ensures that rewards align with the actual utility of the data for enhancing OS agents, incentivizing users to contribute high-quality, diverse, and contextually rich interactions.

### 4.3 Collected Data Information

We deployed our system with 30 registered users, successfully collecting 1M raw action data points (After final data valuation, they were paid more than 12 dollars per hour.) The distributions of action types and application usage within this dataset are presented in Figure 2. Analysis of this data highlights the following:(1) **Click**

**actions** represent the most frequent interaction type across the observed systems. Furthermore, **scrolling operations** are notably more prevalent on macOS compared to Windows. (2) **Office software, Web Browsers, and Coding applications** constitute the top three most frequently used application categories across these systems.

## 5 GUI-Pipe: From Raw Data to Trainable Data

While the CCAgent Net system has collected a significant volume of low-level OS action data from users, this data cannot be directly used to train OS agents. This is primarily because it lacks explicit instructions and labels, and currently, no established unsupervised training methods exist for this type of behavioral data (an interesting direction for future work). To this end, we propose GUI-PIPE, an automatic pipeline that transforms the low-level data the standard instruction tuning data to train OS agent. The pipeline consists of two major process: 1) Interest Region Identification (Section 5.1) and 2) Instruction Generation (Section 5.2). We will also introduce the general statistics of the transformed dataset in Section 5.3.

### 5.1 Interest Region Identification

Each example in the low-level data includes a screenshot and corresponding user action. The user actions are noisy, that it is difficult to directly give the instruction from the click and keyboard input. We turn our sight to the screenshot, which has a lot of interactive elements such as buttons, input box and option lists. To identify these elements, we first employ an off-the-shelf object detection model OmniParser. Given an input screenshot, represented as an image  $I$ , the object detection model, denoted as  $f_{OD}$ , processes  $I$  to identify and localize interactive UI components. The output of this process is a set of detected objects,  $O$ , where each object  $o_k$  includes its bounding box coordinates, a predicted class label, and a confidence score. This can be formalized as:

$$O = \{o_k \mid k = 1, \dots, N\} = f_{OD}(I)$$

Here, each individual detected object  $o_k$  is defined by a tuple:

$$o_k = (b_k, c_k, s_k)$$



- $b_k = (x_{min}^{(k)}, y_{min}^{(k)}, x_{max}^{(k)}, y_{max}^{(k)})$  represents the bounding box coordinates for the  $k$ -th detected interactive element, specifying its top-left and bottom-right corners on the screenshot  $I$ .
- $c_k \in C$  is the predicted class label for the  $k$ -th object, where  $C$  denotes the set of all identifiable interactive element classes (e.g., 'text', 'icon').
- $s_k \in [0, 1]$  is the confidence score, indicating the model's certainty in the detection of  $o_k$ .

Only detections with a confidence score  $s_k$  exceeding a predefined threshold,  $\tau = 0.4$ , in our experiments, are considered valid and passed on for further processing. Currently employed object detector model could classify the type of the object such as "icon" but not the accurate name of the object such as "icon of PowerPoint". Future improvements on fine-grained detection could largely increase the information gain of this stage.

## 5.2 Instruction Generation

After identifying interest regions  $O$  in a screenshot  $I$ , the next challenge is to generate meaningful, region-specific instructions that simulate genuine user intentions. Directly employing a vision-language model [3] (VLM) to generate OS-level commands from these regions often leads to hallucinations and inaccuracies, particularly when non-interactive or static elements are involved. To address these limitations, we propose a two-stage approach: (1) Judgment Rationale Generation, and (2) Final Instruction Generation.

**5.2.1 Judgment Rationale Generation.** The first stage assesses each interest region  $o \in O$  to determine its interactivity and infer its functional role. We present the VLM with a comprehensive context that includes: (i) the full screenshot  $I$  (providing global context), (ii) the cropped image of the interest region  $o$  (providing local context), and (iii) the spatial coordinates of  $o$ , which implicitly connect  $o$  and  $I$  spatially. The VLM uses these three components to reason about the spatial and contextual relevance of  $o$  within the operating system environment. Initially, the VLM assesses whether the region  $o$  is interactive. If deemed interactive, it generates a rationale that describes the element's likely function and usage scenario. Each Multimodal Judgment Rationale is represented as a triplet  $(I, o, r)$ , where  $r$  is the generated rationale. This process enables the system to discard non-interactive or irrelevant regions, ensuring that subsequent instruction generation is both accurate and contextually appropriate.

**5.2.2 Final Instruction Generation.** In the second stage, each validated Multimodal Judgment Rationale  $(I, o, r)$  is transformed into an actionable OS instruction. To support diversity and practical relevance, we manually construct a comprehensive set of instruction examples covering a broad spectrum of typical OS operations. These examples are used as in-context learning (ICL) prompts to guide the VLM in generating coherent and user-friendly operation instructions.

Given  $(I, o, r)$  and a sampled subset of ICL examples, the VLM produces a question-style instruction  $q$  describing the intended action, and a corresponding executable action  $a$ . The interest region  $o$  contributes its bounding box  $b$  to serve as the label. The final

**Table 2: Statistics of all collected datasets.**

| Dataset Split          | Number of Data | Mac:Windows | Click:Scroll:Type |
|------------------------|----------------|-------------|-------------------|
| CCAgent-Raw            | 1M             | 3:7         | 7:2:1             |
| CCAgent-Instruct       | 100M           | 3:7         | 9:0:1             |
| CCAgent-Instruct-Train | 1M             | 3:7         | 9:0:1             |
| CCAgent-Instruct-Test  | 1K             | 5:5         | 9:0:1             |

structured training instance is a triplet  $(I, q, (a, b))$ , consisting of the screenshot, the instruction, and the action-label pair.

Ultimately, our two-stage approach produces high-quality training data triplets, each comprising a screenshot image, an operation instruction, and an associated actionable label.

## 5.3 Dataset Overview

**5.3.1 Data Example.** Below is an example from our generated dataset, showcasing both the system prompt and an instruction tuning instance. The system prompt clearly defines the task objective and constrains the model's output to valid OS actions like click and type. Each instruction comprises a screenshot, a question, and a target action to achieve the question's goal. These actions are either a click or keyboard input. For clicks, we provide the coordinates of the region of interest. For keyboard inputs, we include both coordinates and the content to be typed. Following Qwen2.5-VL's convention, we use absolute pixel coordinates without normalization.

### System Prompt

#### Objective:

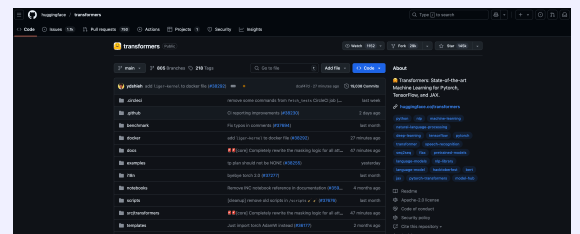
Identify the action to perform(enumerate in [click, keyboard input]) and the exact [x, y] pixel coordinate to interact with on a provided screenshot based on textual instructions.

#### Input Content:

1. Input includes: Screenshot image + Textual action instructions.
2. Instructions to follow (e.g., "Click the blue login button").

The output answer format should be as follows: [action: enum[click, keyboard input], coordinate: [x, y]]. Please strictly follow the format.

### Instruction



Question: <image> Open the setting of the github repo.

Action: [action: click, coordinate: [1113, 82] ]

| Model                          | Development |            | Creative    |            | CAD         |            | Scientific  |             | Office      |             | OS          |             | Avg         |
|--------------------------------|-------------|------------|-------------|------------|-------------|------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|
|                                | Text        | Icon       | Text        | Icon       | Text        | Icon       | Text        | Icon        | Text        | Icon        | Text        | Icon        |             |
| SeeClick [6]*                  | 0.6         | 0.0        | 1.0         | 0.0        | 2.5         | 0.0        | 3.5         | 0.0         | 1.1         | 0.0         | 2.8         | 0.0         | 1.1         |
| OS-Atlas-4B [34]*              | 7.1         | 0.0        | 3.0         | 1.4        | 2.0         | 0.0        | 9.0         | 5.5         | 5.1         | 3.8         | 5.6         | 0.0         | 3.7         |
| ShowUI-2B [17]*                | 16.9        | 1.4        | 9.1         | 0.0        | 2.5         | 0.0        | 13.2        | 7.3         | 15.3        | 7.5         | 10.3        | 2.2         | 7.7         |
| CogAgent-18B [11]*             | 14.9        | 0.7        | 9.6         | 0.0        | 7.1         | 3.1        | 22.2        | 1.8         | 13.0        | 0.0         | 5.6         | 0.0         | 7.7         |
| Aria-GUI [35]*                 | 16.2        | 0.0        | 23.7        | 2.1        | 7.6         | 1.6        | 27.1        | 6.4         | 20.3        | 1.9         | 4.7         | 0.0         | 11.3        |
| OS-Atlas-7B [34]*              | 33.1        | 1.4        | 28.8        | 2.8        | 12.2        | 4.7        | 37.5        | 7.3         | 33.9        | 5.7         | 27.1        | 4.5         | 18.9        |
| Qwen-VL-7B [2]*                | 0.0         | 0.0        | 0.0         | 0.0        | 0.0         | 0.0        | 0.7         | 0.0         | 0.0         | 0.0         | 0.0         | 0.0         | 0.1         |
| Qwen2-VL-7B [2]*               | 2.6         | 0.0        | 1.5         | 0.0        | 0.5         | 0.0        | 6.3         | 0.0         | 3.4         | 1.9         | 0.9         | 0.0         | 1.6         |
| Qwen2.5-VL-3B [2]*             | 14.9        | 2.1        | 20.2        | 1.4        | 4.1         | 4.7        | 34.0        | 7.3         | 22.0        | 3.8         | 6.5         | 2.2         | 11.8        |
| UGround-7B [38]*               | 26.6        | 2.1        | 27.3        | 2.8        | 14.2        | 1.6        | 31.9        | 2.7         | 31.6        | 11.3        | 17.8        | 0.0         | 16.5        |
| UI-R1-3B* [19]                 | 22.7        | 4.1        | 27.3        | 3.5        | 11.2        | 6.3        | 42.4        | 11.8        | 32.2        | 11.3        | 13.1        | 4.5         | 17.8        |
| UI-TARS-2B [29] <sup>#</sup>   | <b>47.4</b> | 4.1        | <b>42.9</b> | <b>6.3</b> | 17.8        | 4.7        | 56.9        | 17.3        | 50.3        | <b>17.0</b> | 21.5        | 5.6         | 27.7        |
| Qwen2.5-VL-3B [2] <sup>†</sup> | 18.2        | 2.1        | 23.2        | 1.4        | 16.8        | 3.1        | 22.2        | 2.7         | 36.2        | 9.4         | 13.1        | 1.1         | 14.7        |
| CCAGENT-GUI-3B <sup>†</sup>    | 38.3        | <b>6.2</b> | 38.9        | 4.9        | <b>42.1</b> | <b>6.3</b> | <b>56.9</b> | <b>21.8</b> | <b>64.4</b> | 15.1        | <b>32.7</b> | <b>11.3</b> | <b>33.7</b> |

**Table 3: Accuracy on different categories of ScreenSpot-Pro.** <sup>†</sup> denotes our implemented models. \* denotes scores reported in [19]. The optimal results are bolded. **CCAgent-GUI-3B** tops in most sub-scores.

**Table 4: Overall results of In-domain and OOD experiments.**

| Model            | In-Domain    |              | ScreenSpot-Pro |              |              |
|------------------|--------------|--------------|----------------|--------------|--------------|
|                  | Mac          | Windows      | Mac            | Windows      | Linux        |
| Qwen2.5VL-7B [2] | 0.580        | 0.621        | 0.314          | 0.221        | 0.140        |
| Qwen2.5VL-3B [2] | 0.448        | 0.490        | 0.174          | 0.128        | 0.080        |
| CCAGENT-GUI-3B   | <b>0.854</b> | <b>0.870</b> | <b>0.349</b>   | <b>0.318</b> | <b>0.160</b> |
| $\Delta$         | (+90.6%)     | (+77.6%)     | (+100.5%)      | (+148.4%)    | (+100.0%)    |

**5.3.2 Data Distribution.** After processing the collected low-level data with GUI-Pipe, we generated over 100 million instruction tuning data points. Due to computational resource limitations, we were unable to train on the entire dataset. Therefore, we randomly sampled 1 million data points for our training set. Apart from this training set, we manually filtered 1,000 examples without screenshot overlapping to serve as our in-domain test set. Table 2 shows the statistics of different splits collected in all of our experiments.

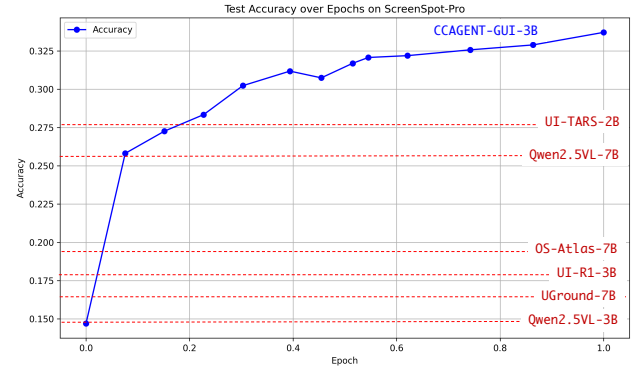
## 6 Experiments

This section provides details on the experimental setup (Section 6.1), results for both in-domain and out-of-domain benchmarks (Section 6.2), and our discussions on the scaling behavior of OS agent training (Section 6.3).

### 6.1 Experimental Setup

#### 6.1.1 Dataset Collection.

**GUI-Pipe.** For Interest Region Identification, we utilized OmniParser [18]’s GUI object detector and the Qwen2.5-VL-32B [2] Vision-Language Model. We ran the official inference script (available at <https://github.com/microsoft/OmniParser>), a process that consumed approximately 570 NVIDIA-3090 GPU hours. During Instruction Generation, we used the vLLM [15] engine to serve Qwen-2.5-VL-32B. For Judgment Rationale Generation, we disabled



**Figure 5: Average accuracy on OOD test during training.**

output sampling to maximize the accuracy of the rationales. Finally, for the instructions, we sampled three outputs per judgment rationale, using top-p=0.99 and temperature=1.0 to enhance data diversity. This step took around 2,600 NVIDIA-H100 GPU hours in total to generate the entire 100M CCAgent-Instruction dataset.

#### 6.1.2 Training and Evaluation.

**Training Details.** We initialized our model with Qwen2.5-VL-3B, a state-of-the-art open-source VLM for its size. During training, we employed a global batch size of 8 and a gradient accumulation step of 2. We trained the model on a 1M training set for one epoch, utilizing a cosine learning rate schedule with an initial learning rate of 1e-5 and a 5% warmup ratio. Notably, the model retained the original screenshot shape rather than resizing images to a fixed size. The total training time amounted to approximately 1,300 NVIDIA-H100 GPU hours, resulting in our CCAgent-GUI-3B model.

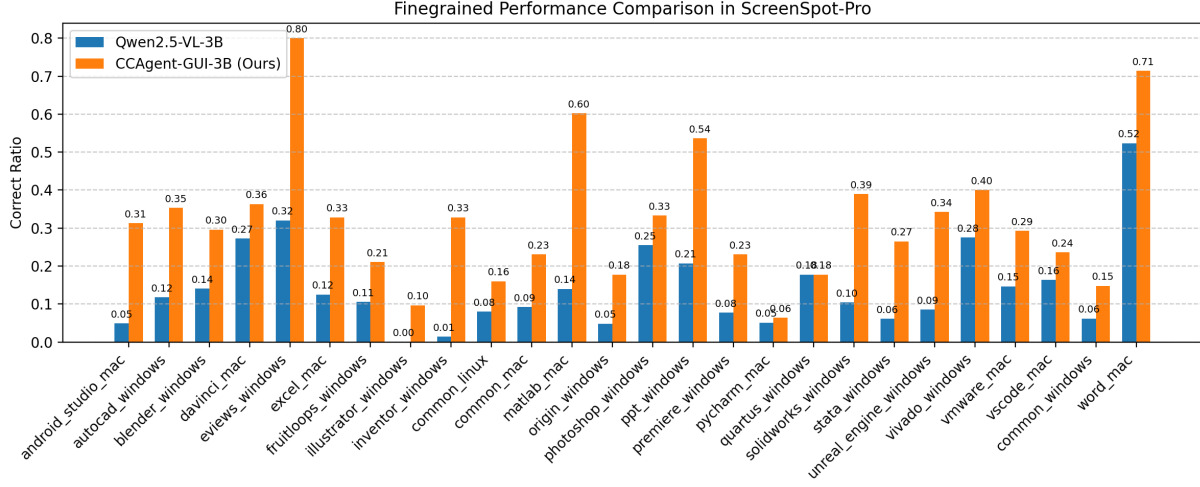


Figure 6: Software sub-scores comparison of ScreenSpot-Pro.

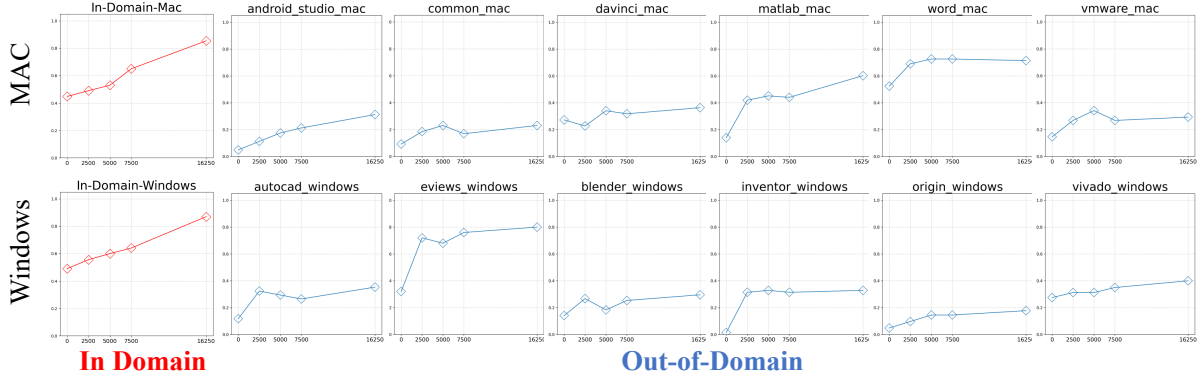


Figure 7: Comparison of data scaling behaviors for tasks across domains and applications.

**Benchmarks.** We evaluate our models on both in-domain and out-of-domain benchmarks. Our in-domain benchmark is our CCAgent-Instruction-Test dataset. It consists of 1,000 examples, equally split between Windows and Mac operating systems. We’ve ensured that this test set has no screenshot overlap with the training set to prevent data leakage. For the out-of-domain benchmark, we use ScreenSpot-Pro [16]. This is a most widely considered challenging OS Agent dataset due to its high application and instruction diversity, as well as high screenshot resolution. ScreenSpot-Pro contains 1,581 examples, broken down by operating system: 927 from Windows, 604 from Mac, and 50 from Linux. These test samples are further categorized into six application domains: "Development," "Creative," "CAD," "Scientific," "Office," and "OS." Each domain features two question types: "text" and "icon." "Text" questions pertain to textual information, while "icon" questions relate to icons that lack associated text. Each sample includes a screenshot, an instruction, and a target interaction area bounding box. During evaluation, the model receives a screenshot and an instruction to predict the coordinates of the interaction target area. A prediction is considered

correct if the predicted coordinates fall within the ground truth target area bounding box.

**Baselines.** We use Qwen2.5-VL-3B as the primary baseline. This model is part of the Qwen-VL series [2], renowned for its versatile vision-language capabilities. We also evaluate its larger variant, Qwen2.5VL-7B [2], as another key comparative model. The overall performance of these primary Qwen2.5VL baselines on both in-domain and out-of-domain benchmarks is presented in Table 4.

For a more fine-grained analysis, particularly on the challenging ScreenSpot-Pro benchmark, we conduct a broader comparison against additional established models, with detailed category-wise results shown in Table 3. Notable among these further baselines are UI-R1-3B [19], which enhances GUI agent action prediction through reinforcement learning, and UI-TARS-2B [27], a model focused on pioneering automated GUI interaction with native agents. Table 3 also includes comparative results for SeeClick, OS-Atlas (4B and 7B variants), ShowUI-2B, CogAgent-18B, Aria-GUI, UGround-7B, other Qwen variants (Qwen-VL-7B, Qwen2-VL-7B). The scores for these models, as presented in Table 3, are sourced from [19].



## 6.2 Results

**6.2.1 Overall Performance Comparisons.** As shown in Table 4, our model significantly outperforms both the Qwen2.5-VL-3B and the larger Qwen2.5-VL-7B baselines, achieving over 100% relative improvement. The performance gain is notably larger on the out-of-domain ScreenSpot-Pro benchmark (+116.3%) than on our in-domain test (+84.1%). This strong generalization, especially on the Linux subset for which we had no training data, validates the effectiveness of our data collection and preprocessing pipeline for creating general-purpose GUI agents.

**6.2.2 Fine-grained Performance Analysis.** As detailed in Table 3, our CCAgent-GUI-3B model achieves a new state-of-the-art average score on the ScreenSpot-Pro benchmark, outperforming similarly sized and larger GUI-specific models like UI-TARS-2B and UGround-7B. Our model shows improvement over the baseline in all sub-categories, although icon recognition remains a significant challenge for Vision-Language Models (VLMs) compared to text recognition.

Figure 6 further reveals that our model improves performance on nearly every application, including many not seen during training. For instance, it scores 0.8 on the ‘evIEWS\_windows’ application with no training data, demonstrating a powerful ability to transfer knowledge across different applications and operating systems. This strongly indicates that our data collection method is effective for developing a truly general-purpose agent.

## 6.3 On the Scaling of OS-Agent Training

Scaling properties are of paramount importance when conducting training for large language models, significantly influencing their eventual performance and capabilities. To better understand the training process and identify potential weaknesses and areas for improvement, we delve into the scaling behavior observed during the training of our model. This analysis aims to shed light on how performance metrics evolve with computational resources, thereby guiding future research and development towards more efficient and powerful OS-Agent systems.

**6.3.1 The overall performance scales well with more data.** As shown in Figure 5, our experiments reveal a positive correlation between the quantity of training data and the overall OOD generalization performance of the OS-Agent. This suggests that expanding the dataset further could lead to more robust and accurate models, affirming the value of large-scale data collection efforts of CCAgent.

**6.3.2 Not all application scales a like.** We further decompose the scaling behaviors of different applications, as shown in Figure 7. This analysis compares between in-domain and out-of-domain tests, along with a breakdown across applications within the out-of-domain tests. The results reveal two interesting findings:

1) **Varying Sensitivity to Training Steps:** Different applications exhibit diverse sensitivities to the number of training steps. Some applications show rapid performance gains early in training, quickly reaching a plateau (e.g., autocad\_windows, blender\_windows, inventor\_windows), while others demonstrate more gradual but consistent improvements over a longer training duration (e.g., android\_studio\_mac, matlab\_mac). This suggests that optimal training

schedules may vary significantly across different application domains and the collected CCAgent has different impact on different application tasks.

2) **In-Domain vs. Out-of-Domain Generalization Gaps:** While both in-domain and out-of-domain performance generally improve with more training, their rates and magnitudes of improvement differ. The in-domain test appears less saturated in the later training stages compared to the out-of-domain test as shown in Figure 5. We believe this is because the model dedicates a substantial portion of its capacity to the in-domain data. Investigating how model size influences the saturation is a promising direction for future research.

### 6.3.3 Limitations and Future Work.

**Dimensions of Scaling.** Due to constraints in training resources, our investigation focused on the data dimension of scaling. We did not thoroughly explore how model performance scales with increasing model size. Future research, with access to greater computational power, should delve into this aspect to provide a more comprehensive understanding of OS-Agent scaling properties.

**Data Coverage VS. More general knowledge.** While collecting more data generally improves performance, there’s a distinction between simply increasing data coverage and instilling more general knowledge in the model. Our current approach largely focuses on expanding the breadth of data. However, for true general-purpose OS-Agents, it’s also essential to explore methods that not only cover more specific scenarios but also enable the model to infer and apply knowledge to entirely novel and unseen contexts. We leave this direction for future explorations.

**Information Gain of Data.** As the system evolves, we aim to integrate a feedback loop from the OS agent training process. Trajectories that lead to a greater reduction in model loss or significant improvement in agent performance could dynamically receive higher valuation scores. We leave this for future exploration.

## 7 Conclusion

This work addresses the critical data scarcity impeding the advancement of Operating System Agents by introducing CCAgent, a novel system for collecting and processing vast amounts of real-world user interaction data. Through its CCAgent-Net distributed data collection component and the GUI-Pipe automated processing pipeline, we successfully gathered over 100 million instruction tuning data instances from 1 million low-level operations. Fine-tuning a relatively small OS agent model with a subset of this data yielded substantial performance improvements (over 100%) on both in-domain and out-of-domain benchmarks, surpassing larger models trained on extensive in-house data, validating the efficacy of the CCAgent system and the GUI-Pipe pipeline.

## 8 Acknowledgments

We thank all reviewers for their valuable suggestions for this work. This work is supported by the National Science Foundation of China under Grant No.61876004.

## 9 GenAI Usage Disclosure

The authors utilized Generative AI (e.g., ChatGPT) exclusively for the purpose of refining grammar and syntax within this document. The substantive work, including all novel intellectual contributions, is an accurate representation of the authors' original efforts and is not primarily a product of GenAI's generative outputs.

## References

- [1] [n. d.]. Claude 3.7 Sonnet System Card. <https://api.semanticscholar.org/CorpusID:276612236>
- [2] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. 2023. Qwen-VL: A Frontier Large Vision-Language Model with Versatile Abilities. *ArXiv preprint abs/2308.12966* (2023).
- [3] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. 2025. Qwen2.5-VL Technical Report. *arXiv preprint arXiv:2502.13923* (2025).
- [4] Patrick Burns, Niklas Etemadfar, Matthew E. Taylor, and Yu Su. 2024. MobileSafetyBench: Evaluating Safety of Mobile Control Agents. *arXiv:2405.10323 [cs.AI]* <https://arxiv.org/abs/2405.10323>
- [5] Liang Chen, Zekun Wang, Shuhuai Ren, Lei Li, Haozhe Zhao, Yunshui Li, Zefan Cai, Hongcheng Guo, Lei Zhang, Yizhe Xiong, Yichi Zhang, Ruoyu Wu, Qingxiu Dong, Ge Zhang, Jian Yang, Lingwei Meng, Shujie Hu, Yulong Chen, Junyang Lin, Shuai Bai, Andreas Vlachos, Xu Tan, Minjia Zhang, Wen Xiao, Aaron Yee, Tianyu Liu, and Baobao Chang. 2024. Next Token Prediction Towards Multimodal Intelligence: A Comprehensive Survey. *arXiv:2412.18619 [cs.CL]* <https://arxiv.org/abs/2412.18619>
- [6] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. 2024. SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. *arXiv:2401.10935 [cs.HC]* <https://arxiv.org/abs/2401.10935>
- [7] Common Crawl Foundation. 2007–2024. Common Crawl. <https://commoncrawl.org>.
- [8] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a Generalist Agent for the Web. *arXiv:2306.06070 [cs.CL]* <https://arxiv.org/abs/2306.06070>
- [9] Yu Fu, Sijie Chen, Hong Huang, Guangsheng Bao, Jingqi Yao, Zhicheng Yang, Ziyu Guo, Yanbin Jiang, Pengfei Liu, and Weihang Guo. 2024. GUI-WORLD: A Dataset and Benchmark for Evaluating GUI Task Execution Agents. *arXiv:2405.15881 [cs.HC]* <https://arxiv.org/abs/2405.15881>
- [10] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. 2024. WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models. *arXiv:2401.13919 [cs.CL]* <https://arxiv.org/abs/2401.13919>
- [11] Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazhen Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxuan Zhang, Juanzi Li, Bin Xu, Yuxiao Dong, Ming Ding, and Jie Tang. 2024. CogAgent: A Visual Language Model for GUI Agents. *arXiv:2312.08914 [cs.CV]* <https://arxiv.org/abs/2312.08914>
- [12] Xueyu Hu, Tao Xiong, Biao Yi, Zishu Wei, Ruixuan Xiao, Yurun Chen, Jiasheng Ye, Meiling Tao, Xiangxin Zhou, Ziyu Zhao, et al. 2024. OS Agents: A Survey on MLLM-based Agents for Computer, Phone and Browser Use.
- [13] Xueyu Hu, Tao Xiong, Biao Yi, Zishu Wei, Ruixuan Xiao, Yurun Chen, Jiasheng Ye, Meiling Tao, Xiangxin Zhou, Ziyu Zhao, et al. 2024. OS Agents: A Survey on MLLM-based Agents for Computer, Phone and Browser Use.
- [14] Noam Kahlon, Guy Rom, Anatoly Efros, Filippo Galgani, Omri Berkovitch, Sapir Caduri, William E. Bishop, Oriana Riva, and Ido Dagan. 2025. Agent-Initiated Interaction in Phone UI Automation. In *Companion Proceedings of the ACM on Web Conference 2025* (Sydney NSW, Australia) (WWW '25). Association for Computing Machinery, New York, NY, USA, 2391–2400. doi:10.1145/3701716.3717526
- [15] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
- [16] Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. 2025. ScreenSpot-Pro: GUI Grounding for Professional High-Resolution Computer Use. *arXiv:2504.07981 [cs.CV]* <https://arxiv.org/abs/2504.07981>
- [17] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Weixian Lei, Lijuan Wang, and Mike Zheng Shou. 2024. ShowUI: One Vision-Language-Action Model for GUI Visual Agent. *arXiv:2411.17465 [cs.CV]* <https://arxiv.org/abs/2411.17465>
- [18] Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. 2024. OmniParser for Pure Vision Based GUI Agent. *arXiv:2408.00203 [cs.CV]* <https://arxiv.org/abs/2408.00203>
- [19] Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Guanqing Xiong, and Hongsheng Li. 2025. UI-R1: Enhancing Action Prediction of GUI Agents by Reinforcement Learning. *arXiv:2503.21620 [cs.AI]* <https://arxiv.org/abs/2503.21620>
- [20] Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Guanqing Xiong, and Hongsheng Li. 2025. UI-R1: Enhancing Efficient Action Prediction of GUI Agents by Reinforcement Learning. *arXiv:2503.21620 [cs.AI]* <https://arxiv.org/abs/2503.21620>
- [21] Yao Ma, Hongyu Ren, Zihan Wang, Jialu Liu, Ziyi Zhou, Zhitao He, Huazheng Wang, and Jiliang Tang. 2024. Explorer: Synthesizing Goal-Oriented Web Trajectories using Large Language Models. *arXiv:2402.17098 [cs.AI]* <https://arxiv.org/abs/2402.17098>
- [22] Kai Mei, Xi Zhu, Wujiang Xu, Wenye Hua, Mingyu Jin, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. 2025. AIOS: LLM Agent Operating System. *arXiv:2403.16971 [cs.OS]* <https://arxiv.org/abs/2403.16971>
- [23] Satoshi Nakamoto and A Bitcoin. 2008. A peer-to-peer electronic cash system. *Bitcoin*. URL: <https://bitcoin.org/bitcoin.pdf> 4, 2 (2008), 15.
- [24] Reichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Benjamin Chess, and John Schulman. 2022. WebGPT: Browser-assisted question-answering with human feedback. *arXiv:2112.09332 [cs.CL]* <https://arxiv.org/abs/2112.09332>
- [25] Dang Nguyen, Jian Chen, Yu Wang, Gang Wu, Namyong Park, Zhengmian Hu, Hanjia Lyu, Junda Wu, Ryan Aponte, Yu Xia, Xintong Li, Jing Shi, Hongjie Chen, Viet Dac Lai, Zhouhang Xie, Sungchul Kim, Ruiyi Zhang, Tong Yu, Mehrab Tanjim, Nesreen K. Ahmed, Puneet Mathur, Seunghyun Yoon, Lina Yao, Branislav Kveton, Thien Huu Nguyen, Trung Bui, Tianyi Zhou, Ryan A. Rossi, and Franck Dernoncourt. 2024. GUI Agents: A Survey. *arXiv:2412.13501 [cs.AI]* <https://arxiv.org/abs/2412.13501>
- [26] OpenAI. 2022. Introducing ChatGPT. (2022).
- [27] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. *arXiv:2307.16789 [cs.AI]*
- [28] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. 2025. UI-TARS: Pioneering Automated GUI Interaction with Native Agents. *arXiv preprint arXiv:2501.12326* (2025).
- [29] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjun Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, Xiaojun Xiao, Kai Cai, Chuang Li, Yaowei Zheng, Chaolin Jin, Chen Li, Xiao Zhou, Minchao Wang, Haoli Chen, Zhaojian Li, Haihua Yang, Haifeng Liu, Feng Lin, Tao Peng, Xin Liu, and Guang Shi. 2025. UI-TARS: Pioneering Automated GUI Interaction with Native Agents. *arXiv:2501.12326 [cs.AI]* <https://arxiv.org/abs/2501.12326>
- [30] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. 2023. Android in the Wild: A Large-Scale Dataset for Android Device Control. *arXiv:2307.10088 [cs.LG]* <https://arxiv.org/abs/2307.10088>
- [31] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *ArXiv preprint abs/2302.13971* (2023).
- [32] Hang Wang, Ruibo Liu, Shangqing Tu, Peng Liu, Ziru Chen, Jason Liu, Dakuo Wang, Salla Franzén, Peter Alexander Gloor, Yasheng Wang, Xin Wang, and Yining Wang. 2024. CRAB: A Comprehensive Benchmark for Cross-Environment Agent Behavior Evaluation. *arXiv:2405.07703 [cs.AI]* <https://arxiv.org/abs/2405.07703>
- [33] Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. 2024. OS-Copilot: Towards Generalist Computer Agents with Self-Improvement. *arXiv:2402.07456 [cs.AI]* <https://arxiv.org/abs/2402.07456>
- [34] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. 2024. OS-ATLAS: A Foundation Action Model for Generalist GUI Agents. *arXiv:2410.23218 [cs.CL]* <https://arxiv.org/abs/2410.23218>
- [35] Yuhao Yang, Yue Wang, Dongxu Li, Ziyang Luo, Bei Chen, Chao Huang, and Junnan Li. 2024. Aria-UI: Visual Grounding for GUI Instructions. *arXiv:2412.16256 [cs.HC]* <https://arxiv.org/abs/2412.16256>
- [36] Boyuan Zhang, Peter Li, Jingkan Yang, Zhiyong Wu, Zhidong Gao, Feng Gao, Tiancheng Jia, Sijia Chen, Yunfei Chu, Tian Ouyang, Yijing Shi, Letian Zhang, Ziyu Zhang, Xinyu Wang, Jingtong Zhang, Lin Zhao, Shengyao Tong, Junda Liu, Pengfei Liu, Yin Zhang, and Zhou Jun. 2024. OmniACT: A Dataset and Benchmark for Enabling Multimodal Generalist Autonomous Agents for Desktop

- and Web. arXiv:2402.17553 [cs.AI] <https://arxiv.org/abs/2402.17553>
- [37] Chaoyun Zhang, Liqun Li, Shilin He, Xu Zhang, Bo Qiao, Si Qin, Minghua Ma, Yu Kang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. 2024. UFO: A UI-Focused Agent for Windows OS Interaction. arXiv:2402.07939 [cs.HC] <https://arxiv.org/abs/2402.07939>
- [38] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. 2024. UGround: Grounding Through Understanding in UI Navigation. arXiv:2410.05243 [cs.CV] <https://arxiv.org/abs/2410.05243>
- [39] Wenyu Zhong, Zheng-Jun Zha, Zhaohua Chen, Min Zhou, Zeren Zhang, and Ting Zhang. 2023. Insight: A Comprehensive Dataset and Benchmark for GUI Modeling and Automation. arXiv:2305.16409 [cs.SE] <https://arxiv.org/abs/2305.16409>